VERITAS: Formally Verified, Competence-Aware Interpretable Guardrails for Language Models

Anwar Debes
University of Agder
Draft v1.0, June 2026

Abstract

Every deployed guardrail for language-model systems is validated the same way: by red-teaming. A team throws adversarial prompts at it, and if enough bounce off, it ships. But passing N probes proves nothing about the $N{+}1$-th, and a learned guardrail fails *silently* off-distribution: it keeps emitting confident verdicts on inputs it has no basis to judge. We argue that an *interpretable* guardrail need not be checked this way, because a guardrail whose decision is a propositional formula is *machine-verifiable*: an exact reasoner can settle a safety question about *all* inputs at once. We present **VERITAS**, a formally verified, competence-aware interpretable guardrail built on a Tsetlin Machine. We do *not* claim to invent Tsetlin-Machine verification; the exact Boolean encoding and robustness/equivalence checking are due to Przybysz et al. [2023], and VERITAS is the verified-guardrail *system* they enable. It contributes (i) safety-property classes a guardrail actually needs: *no-bypass invariants*, *trigger-necessity* (no alarm without an audited cue), and *monotonicity* as a soundness check, each decided exactly with a proof or a concrete counterexample; (ii) *constructive* verification-guided repair, because we show you cannot *train* a guard to a proof (empirical counterexample retraining never reaches, and cannot certify, zero bypasses), while a constructive clause repair closes every one by construction; (iii) a *verified-competence envelope* that abstains, provably, on inputs outside the region where a guarantee holds, turning silent distribution-shift failure into auditable escalation; and (iv) a tamper-evident, *prover-independent* assurance certificate. Across two real datasets, three independent backends agree on 17/17 cross-checked queries; verification decides cases red-teaming cannot (it certifies 0.09 of obligations *robust*, which no finite red team can); constructive repair reaches 0.0 provable bypasses where retraining is stuck at 8.0, and under a 0.899→0.505 distribution-shift collapse the envelope escalates silently-missed attacks behind a *provably-sound* abstention boundary (sound in 1.00 of runs). VERITAS is the fourth pillar of an interpretable agent-safety line after THGTM, TRACE and REINS, and the one that makes the guard itself an object of proof.

1 Introduction

A guardrail is a classifier that sits between a user and a language model and decides whether a prompt, a retrieved document, or a model output is safe to pass. The dominant designs, Llama Guard [Inan et al., 2023], ShieldGemma [Zeng et al., 2024], WildGuard [Han et al., 2024], Constitutional Classifiers [Sharma et al., 2025], are themselves multi-billion-parameter neural networks, and they inherit two properties that should worry anyone deploying them as a *safety* component.

They are validated by spot-checking. The evidence that a guardrail is safe is, almost universally, a benchmark: an attack-success-rate on a fixed set of adversarial prompts. But an attacker is not restricted to the benchmark. Universal and transferable jailbreaks [Zou et al., 2023] are discovered *after* each defense ships precisely because surviving a finite test set is not a

guarantee about the infinite space of inputs. Red-teaming can *find* a bypass; it can never *certify* that none exists. For a safety component this is the wrong kind of evidence.

They fail silently. A learned classifier has a competence boundary, and the dangerous failure is the quiet one: under distribution shift it does not announce "I am out of my depth"; it just keeps returning confident, wrong verdicts. In our own measurements a guard that scores 0.899 in-domain collapses to 0.505, near the base rate, on a sibling dataset, while never signalling the loss of competence.

The thesis of this paper is that an *interpretable* guardrail does not have to accept either limitation. If the guard's decision is a Boolean formula over named features, then it is not merely *readable*: it is *machine-verifiable*. An exact reasoner (a SAT or SMT solver) can answer questions about *all* inputs at once: *is there any input containing this attack phrase that the guard waves through?* The answer is a proof, or a concrete counterexample. This is the transparency dividend of propositional models, and it is unavailable to a dense network, for which the same questions are intractable and only ever approximated.

What is, and is not, new. The enabling step, compiling a trained Tsetlin Machine into exact Boolean form and verifying robustness or equivalence with a solver, is *not* our contribution. It is due to Przybysz et al. [2023] (with the Tsetlin Machine's inventor as a co-author), who give the encoding, prove it correct, and verify ϵ-robustness, equivalence and similarity on image and text classifiers. We build directly on that result. Our contribution is everything needed to turn it into a *verified guardrail*, and it is a conjunction, not a single trick:

1. **Guardrail property classes** (§4). Robustness and equivalence are not the questions a deployer asks. We add *no-bypass invariants* ("every input matching this attack template is flagged"), *trigger-necessity* ("the guard never raises an alarm without an audited cue token", the property a spurious shortcut violates), and *monotonicity* as a soundness check ("a known-bad token can never be evidence of safety"). Each compiles to one exact query.
2. **Constructive repair** (§5). We find, and prove, that you cannot *train* a guard to satisfy a hard universal property: empirical counterexample-retraining never reaches zero provable bypasses and can even *lose* a guarantee it had. Because the guard is propositional, we instead *construct* the fix, appending the minimal clause set that provably enforces the obligation.
3. **Verified-competence abstention** (§6). A verified TRIGGER-NECESSITY property defines a region where the guard's verdict is backed by an audited cue; outside it the guard is guessing. The envelope abstains there, and the abstention boundary is itself *verified*, unlike a softmax or margin threshold.
4. **A prover-independent assurance certificate** (§7). The verification record is sealed into a tamper-evident, hash-chained log; a refuted property carries a counterexample an auditor re-checks with pure arithmetic, no solver and no trust in ours.

VERITAS is the fourth component of an interpretable agent-safety line, THGTM [Debes, 2026b] (the learned substrate), TRACE [Debes, 2026c] (runtime detection and a certificate of *what was checked*), REINS [Debes, 2026a] (minimal-intervention enforcement), and the piece they lacked: assurance *of the guard itself*. Every number, proof and certificate regenerates from the repository on a single CPU.

2 Background and related work

Verification of interpretable classifiers. Exact verification of discrete models is mature. Binarized neural networks are encoded into SAT and checked for robustness [Narodytska et al.,

2018]; decision trees and rule lists admit SAT/SMT analysis and *formal explanations* [Ignatiev et al., 2019]; neural networks more broadly are handled by specialised SMT solvers [Katz et al., 2017]. For Tsetlin Machines specifically, Przybysz et al. [2023] give the exact propositional encoding and verify ϵ-robustness, equivalence and similarity. VERITAS reuses that encoding wholesale; we contribute the property classes, repair, envelope and certificate around it, and the guardrail framing none of this prior work targets.

Tsetlin Machines. A Tsetlin Machine [Granmo, 2018] learns a set of *clauses*, each a conjunction of literals (a feature or its negation), whose ± 1 votes are thresholded into a class. Because a clause is literally a logical AND of named tokens, a TM text classifier [Yadav et al., 2021] is auditable, and, as we use here, encodable into exact Boolean form. THGTM [Debes, 2026b] contributes the Echo-Trace automaton substrate our guard is built on.

LLM guardrails. The production guardrails cited above are strong detectors but uniformly opaque neural networks validated by benchmark, with no notion of a global guarantee; none is formally verified. VERITAS trades raw detection breadth for a guard small and transparent enough to *prove* things about. We do not claim to beat them on accuracy and report our (deliberately compact) guard's numbers honestly.

Selective prediction. Abstaining when uncertain is classical, Chow's reject option [Chow, 1970], selective classification with coverage guarantees [Geifman and El-Yaniv, 2017, El-Yaniv and Wiener, 2010], and conformal prediction [Angelopoulos and Bates, 2023]. VERITAS does not claim to invent abstention; its envelope differs in that the accept/reject boundary is a *formally verified precondition*, not a score threshold, so the abstention region is itself certifiable.

Counterexample-guided synthesis and shielding. Closing a loop with counterexamples is CEGIS [Solar-Lezama et al., 2006] and CEGAR [Clarke et al., 2000]; enforcing safety at runtime by construction is shield synthesis [Bloem et al., 2015, Alshiekh et al., 2018]. Our constructive repair instantiates this idea on a guard; our finding that the *learning*-based loop does not converge motivates the constructive operator.

Machine-checkable proofs. That a verdict should be checkable without trusting the tool that produced it is the principle behind DRAT proofs for SAT [Wetzler et al., 2014] and certifying algorithms [McConnell et al., 2011]. VERITAS's certificate applies it to guard assurance: a counterexample is re-checked by evaluating the (tiny, included) guard, and a proof of robustness is reproducible by any solver.

Shortcut learning and assurance. Classifiers latch onto spurious lexical cues that vanish under shift [Geirhos et al., 2020, McCoy et al., 2019]; our TRIGGER-NECESSITY property is a verifiable check against exactly this. Finally, the call for *guaranteed-safe* AI [Dalrymple et al., 2024] and the EU AI Act's robustness and accuracy obligations (Art. 15; European Parliament and Council, 2024) frame why a provable, rather than spot-checked, guardrail matters.

3 Problem setting and the guard

The guard. VERITAS verifies a binary Tsetlin-Machine content guard. Text is booleanised by token presence: a vocabulary of F discriminative unigrams/bigrams is selected on the training split, and an input becomes an F-bit vector "token t occurs". A binary TM with C clauses over the $2F$ literals (raw + negated) decides

$$\text{vote}(x) = \sum_j p_j \, c_j(x), \qquad \textsc{unsafe}(x) \iff \text{vote}(x) > 0, \tag{1}$$

where clause j has polarity $p_j \in \{+1, -1\}$ and output $c_j(x)$ equal to the AND of its included literals (an empty clause outputs 0 in inference). The vocabulary is human-readable, so a clause reads `IF ignore AND previous` $\rightarrow$ `UNSAFE`.

Exact encoding. Following Przybysz et al. [2023], the decision (1) compiles exactly into Boolean form: feature bits $b_1, \ldots, b_F$, literals b_i or $\neg b_i$, clause conjunctions, and the ± 1-weighted threshold. We emit it two ways, as an SMT term for z3 [de Moura and Bjørner, 2008] (the vote as an integer) and as a Tseitin [Tseitin, 1983] CNF encoding plus a cardinality constraint for a SAT solver via PySAT [Ignatiev et al., 2018] (the ± 1 threshold becomes $\geq k$ once the negative-polarity clause literals are flipped). We additionally provide brute-force enumeration for small F as an oracle.

Proposition 1 (Faithful encoding)**.** *For every feature vector $x \in \{0,1\}^F$, the encoded vote equals* vote(x) *of* (1)*, including the inference-mode convention that an empty clause contributes* 0*. Hence a property holds of the encoding iff it holds of the guard as deployed.*

The proof is immediate from the construction and is checked empirically to the bit in the test suite (the encoder reproduces the guard's vote on every probe). We verify over the guard's *input space* $\{0,1\}^F$; this is sound for the guard as a function (it consumes feature vectors), and we optionally add realizability constraints so a counterexample corresponds to plausible text.

Threat model. An adversary may submit any text; the corresponding feature vector ranges over $\{0,1\}^F$ (or its realizable subset). VERITAS makes three kinds of claim: a *no-bypass* guarantee (no input of a declared form evades), a *competence* guarantee (the guard abstains outside its verified region), and an *audit* guarantee (the verification record is tamper-evident). We do not defend against attacks outside the declared property set, nor against a compromised verifier host; and verification is over the booleanised feature space, not raw byte strings (§9).

4 Guardrail properties

A *property* is a statement about the guard on every input that compiles to one query "$\exists x$: PRE$(x) \wedge \neg$POST(x)"; UNSAT means the property holds, SAT returns a counterexample. Two everyday safety questions reduce to one primitive, the *invariant* $\forall x$: PRE$(x) \Rightarrow$ POST(x):

No-bypass / must-flag. PRE = "tokens T all present", POST = UNSAFE: every input containing the attack phrase T is flagged. A counterexample is a concrete bypass.

Trigger-necessity (no spurious cue). PRE = "every audited cue in U absent", POST = SAFE: the guard never fires without an audited reason. A counterexample is a *hidden trigger*, an input the guard flags for reasons outside U, the signature of a shortcut that a distribution shift will break.

Two further properties need arithmetic over the vote and so are separate queries: the *exact robustness radius* (the minimum token edits to flip a verdict, Przybysz et al.'s ϵ-robustness, which we reuse) and *monotonicity* ($\forall x$: vote$(x|b_t{=}1) \geq$ vote$(x|b_t{=}0)$ for a known-bad token t: adding t can never make the guard *safer*). *Equivalence* compares two guards over all inputs and certifies that a change had no unintended effect.

5 Constructive repair: you cannot train your way to a proof

Suppose MUST-FLAG(t) is refuted. The natural fix is the counterexample-guided loop: add the counterexample as a labelled (unsafe) example and retrain. We show this does not work for a hard universal property. Retraining only *softly* biases a stochastic learner; across rounds the

count of provable bypasses does not reach zero, and a guarantee that held can be *lost* when the retrain reshapes unrelated clauses (§8, E3). Verification, here, cannot just grade; it must *repair*.

Because the guard is propositional, the repair is constructive.

Proposition 2 (Constructive repair)**.** *Let the guard have N^- negative-polarity clauses. Appending $k = N^- + 1$ positive single-literal clauses $(+1, \{b_t\})$ makes* MUST-FLAG(t) *hold: for any x with b_t=1 the appended clauses contribute $+k$, while the original vote is $\geq -N^-$, so* $\text{vote}(x) \geq -N^- + (N^- + 1) = 1 > 0$.

The repaired object is still a Tsetlin Machine, the appended clauses read `IF` $t \rightarrow$ `UNSAFE`, and re-verification confirms the obligation by construction. For *attack-exclusive* tokens (those absent from benign training text) the accuracy cost is small or favourable (E3). This is the sense in which verification *teaches* the guard: it supplies a sound, re-verifiable repair operator that no amount of retraining can match.

6 Verified-competence abstention

A verified TRIGGER-NECESSITY(U) has a direct operational reading: if no audited cue in U is present, the guard's verdict is SAFE *by absence of evidence*, not by positive judgement. The *verified envelope* treats that region as out of competence and abstains (escalates to a human or a heavier model); inside it, an audited cue is present, the verdict is forwarded.

Proposition 3 (Sound abstention boundary)**.** *If* TRIGGER-NECESSITY(U) *is verified, then every input the envelope answers contains an audited cue from U, and every no-cue input is escalated. Equivalently, the guard contributes no safety signal on the abstained region (it is provably* SAFE *there regardless of the true label), so abstaining loses no detection the guard could have provided.*

Unlike a softmax or margin threshold, the accept/reject boundary is a verified precondition, so the abstention region is certifiable. Under severe distribution shift this converts silent misses into escalations (E4): of the attacks a shifted guard silently passes, the envelope escalates those it had no cue for.

7 The assurance certificate

VERITAS seals the verification record, a guard fingerprint plus, per property, the verdict and any counterexample, into a hash-chained, HMAC-signed log, exactly as TRACE seals decisions:

$$h_i = \text{SHA256}(h_{i-1} \,\|\, \text{canonical}(\text{payload}_i)), \qquad \sigma_i = \text{HMAC}_k(h_i),$$

with a per-session Merkle root [Laurie et al., 2013, Schneier and Kelsey, 1998, Krawczyk et al., 1997].

Proposition 4 (Tamper-evidence)**.** *Under collision resistance of SHA-256, any modification, insertion, deletion or reordering of a sealed certificate breaks a chain link or the Merkle root and is detected by verification.*

The certificate is *prover-independent.* A REFUTED entry carries a counterexample feature vector; an auditor re-evaluates the included guard on it with pure arithmetic, no solver, no trust in ours, and sees the bypass. A VERIFIED entry is reproducible by re-running any backend (we cross-check z3, PySAT and brute force), or by checking the emitted DRAT proof [Wetzler et al., 2014]. HMAC gives symmetric authentication, not non-repudiation; an Ed25519 signature over the Merkle root is the drop-in upgrade.

8 Experiments

Protocol. Guards are trained on two real Apache-2.0 datasets (`deepset/prompt-injections`, `jackhhao/jailbreak`); verification uses z3 and PySAT with brute-force cross-checks. We report mean over 8 seeds. The guard is deliberately *compact*: verifiability favours small models, and quantifying that trade-off is part of the contribution, not a limitation to hide. All latencies are `perf_counter` wall-clock on one CPU.

8.1 E1, Verification works, and the verifier is sound

We verify a property suite on guards across a size frontier ($C \times F$ from 8×24 to 64×96). Verification is practical: a query takes 15.5 ms at the small end and 151 ms at the large, while accuracy rises from 0.620 to 0.800, the interpretability / verifiability / accuracy frontier (Figure 1). Crucially, our *verifier* is itself checked: on every fully cross-checked query, z3, PySAT and brute-force enumeration return the same verdict (17/17 agreement), so a VERITAS verdict does not rest on a single solver.

8.2 E2, Verification decides what red-teaming cannot

For a lexicon of attack tokens we declare MUST-FLAG obligations and compare exact verification with random red-teaming. Verification decides every case: it *certifies* 0.09 of the tokens *robust*, a guarantee no finite red team can produce at any budget, and returns a concrete bypass for the remaining 0.91. Red-teaming, by contrast, can only sample: even a thorough campaign certifies *nothing* robust, and its bypass-finding is budget- and luck-dependent (Figure 2). The asymmetry is the point: a guardrail's safety case needs the negative result, "no bypass exists", and only verification supplies it, in 66 ms per token.

8.3 E3, You cannot train your way to a proof

Starting from 6.8 provable bypasses on attack-exclusive obligations, empirical counterexample-retraining is stuck at 8.0 (it never reaches zero, loses 1.2 previously-held guarantees on average, and drags test accuracy from 0.692 to 0.565). Constructive repair (Prop. 2) instead reaches 0.0 provable bypasses, zero, by construction, while *keeping* accuracy ($0.692 \rightarrow 0.711$), at a cost of 104 appended single-literal clauses (Figure 2). Verification supplies the repair operator the learning loop cannot.

8.4 E4, Verified abstention under distribution shift

A guard trained on jailbreak text scores 0.899 in-domain and collapses to 0.505, near the base rate, on prompt-injection text, *silently*. The TRIGGER-NECESSITY(U) property is verified for the audited lexicon in 1.00 of runs, so the no-cue region, where the guard emits a constant SAFE verdict by absence of evidence, is exactly where a silently-missed attack can hide; the envelope escalates that whole region. At its operating point it answers 0.62 of out-of-domain inputs and recovers 0.30 of the attacks the shifted guard silently passes (Figure 3). We are deliberately honest about what this does *not* buy: MISSED-RECOVERED rewards abstaining more, and an uncertified margin-threshold selective baseline [Geifman and El-Yaniv, 2017] coverage-matched on the same out-of-domain set (0.61 coverage) recovers 0.58, *more* than the envelope. The envelope's contribution is therefore not a higher recovery rate but that its abstention boundary is a *formally verified* one, certifiable to an auditor, which no score threshold can claim. Making a verified boundary also competitive on the selective-accuracy frontier is future work (§9).

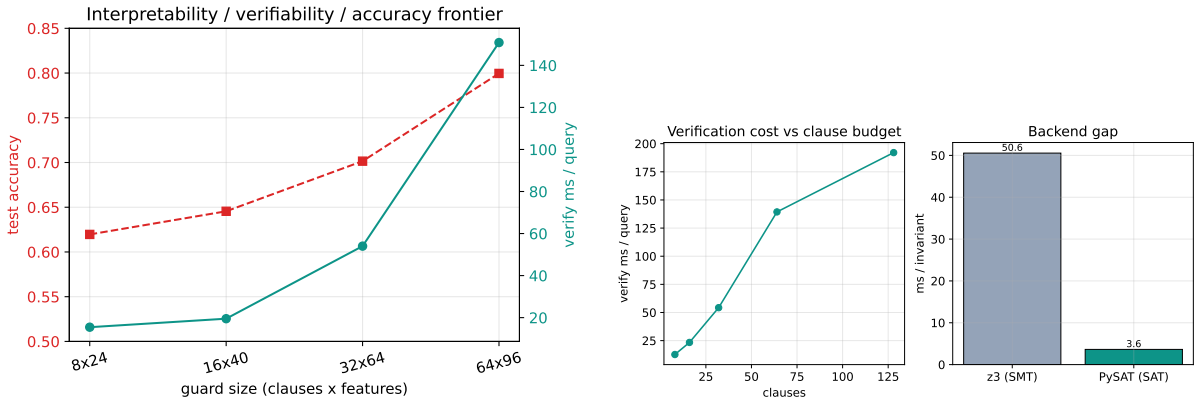


Figure 1: E1/E6: (left) the interpretability/verifiability/accuracy frontier, accuracy and verify-time both rise with guard size; (right) verification time vs clause budget, and the z3-vs-PySAT backend gap.

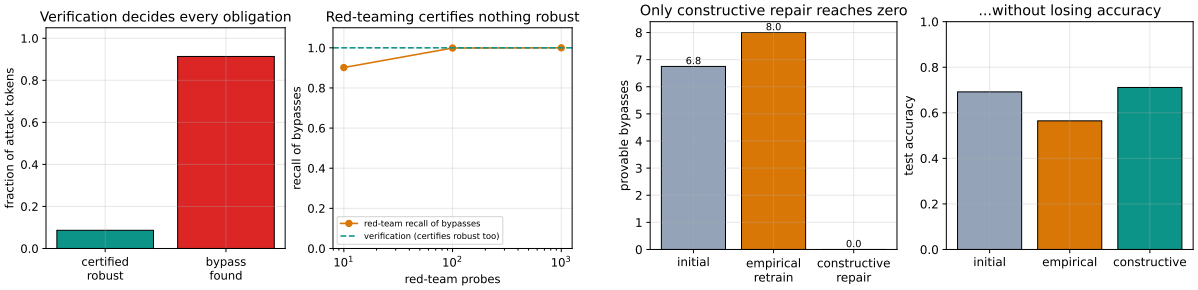


Figure 2: E2/E3: (left) verification certifies robust tokens and finds every bypass; red-teaming certifies none. (right) constructive repair drives provable bypasses to zero where empirical retraining cannot.

8.5 E5, The assurance certificate

We seal certificates over mixed verified/refuted specifications and apply 6 tamper classes plus a signature forgery: all are detected at rate 1.00. Untampered certificates always verify; every counterexample is independently re-evaluable on the included guard (rate 1.00), and an optional full re-run of the verifier reproduces every recorded verdict (rate 1.00). The forged signature, a valid recomputed hash with no key, is caught by the HMAC check (1.00), isolating "cannot re-sign".

8.6 E6, Systems

Verification cost grows with guard size, from 13 ms at C=8 to 192 ms at C=128 (Figure 1, right). Backend choice matters: the SAT encoding via PySAT decides an invariant in 3.6 ms against 50.6 ms for the SMT encoding, an order of magnitude, so we default to SAT for invariants and reserve SMT for the arithmetic properties. By property type, an invariant costs 65 ms, an exact robustness radius 68 ms, and a monotonicity check 107 ms (it needs two guard copies).

9 Discussion and limitations

- **Feature-space verification.** We verify over booleanised feature vectors, the guard's actual input. A counterexample is a feature vector; mapping it back to a worst-case *string* (and modelling tokenisation/obfuscation exactly) is future work, partly addressed by the optional

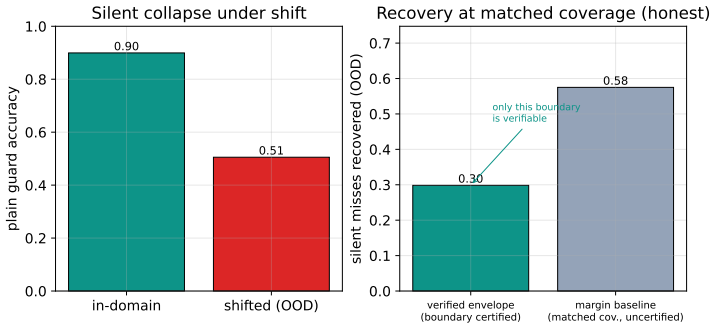


Figure 3: E4: (left) under distribution shift the plain guard collapses silently; (right) the envelope escalates no-cue inputs, recovering some silently-missed attacks. A coverage-matched uncertified margin baseline recovers *more*; the envelope's contribution is that its abstention boundary is the only *verifiable* one (sound in 1.00 of runs), not a higher recovery rate.

realizability constraints.

- **Compact guards.** Verifiability favours small models; our guards trade accuracy for provability and are not competitive with billion-parameter detectors on raw accuracy. The frontier (E1) makes the trade-off explicit. Verification cost grows with size and some large-guard queries would exceed a practical budget.
- **Declared properties only.** VERITAS proves what someone wrote a property for; a harmful behaviour nobody specified is not covered. Trigger-necessity mitigates this by catching *unspecified* reliance, but property mining is open.
- **Repair changes behaviour.** Constructive repair forces a verdict on a declared token set; for non-attack-exclusive tokens it raises false positives. We report the cost and recommend it for attack-exclusive obligations.
- **Symmetric authentication.** The certificate is HMAC-based; non-repudiation needs an asymmetric signature over the Merkle root.

10 Ethics and broader impact

This work is defensive and uses public, Apache-2.0 data with no personal information. A verified guardrail reduces the false confidence of spot-checked safety and gives auditors machine-checkable evidence, supporting emerging obligations [European Parliament and Council, 2024, Dalrymple et al., 2024]. The interpretable clauses that make verification possible also reveal the guard's logic to an adversary; we judge the transparency benefit to outweigh this, consistent with open-source security practice, and note that a *verified* guard is precisely one whose exposed logic has no bypass to exploit for the declared class.

11 Reproducibility

The repository contains the library, the unit tests (including the bit-exact encoder check and the three-backend agreement), the six experiments, the figure and number scripts, the vendored datasets, and this source. `make test` runs the suite; `make reproduce` regenerates every `results/*.json`; `make paper` compiles this PDF. The verifier's soundness is itself part of the test suite.

12 Conclusion

A guardrail is a safety component, and a safety component deserves a proof, not a spot-check. VERITAS shows that when the guard is interpretable, a propositional Tsetlin Machine, proof is available: we can decide, over all inputs, whether an attack template bypasses it; we can repair the bypass by construction when it does; we can make the guard abstain, provably, where it has no competence; and we can seal the whole assurance argument into a certificate an auditor checks without trusting us. The exact verification machinery is not ours but Przybysz et al.'s; the verified, competence-aware guardrail it makes possible is, we believe, new, and it is the natural completion of an interpretable agent-safety line whose other pillars detect, certify and enforce. What remains is string-level verification, property mining, and scaling the frontier, each a concrete step toward guardrails that are not merely tested but *accountable*.

References

Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1708.08611. arXiv:1708.08611.

Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *Foundations and Trends in Machine Learning*, 2023. URL https://arxiv.org/abs/2107.07511. arXiv:2107.07511.

Roderick Bloem, Bettina Könighofer, Robert Könighofer, and Chao Wang. Shield synthesis: Runtime enforcement for reactive systems. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2015. URL https://link.springer.com/chapter/10.1007/978-3-662-46681-0_51.

C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV)*, 2000. URL https://link.springer.com/chapter/10.1007/10722167_15.

David Dalrymple, Joar Skalse, Yoshua Bengio, Stuart Russell, Max Tegmark, Sanjit Seshia, et al. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024. URL https://arxiv.org/abs/2405.06624.

Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008. URL https://link.springer.com/chapter/10.1007/978-3-540-78800-3_24.

Anwar Debes. Reins: Model-free, certifying least-cost repair of LLM-agent tool-call trajectories, 2026a. Reference implementation, v1.0.

Anwar Debes. Thgtm: A temporal hierarchical graph tsetlin machine with echo-trace automata for trajectory-level verification of agentic ai, 2026b. Reference implementation, v0.1.

Anwar Debes. Trace: A tsetlin runtime assurance and certification engine for interpretable, tamper-evident guardrails on agentic ai, 2026c. Reference implementation, v1.0.

Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.

European Parliament and Council. Regulation (eu) 2024/1689 (the EU AI act), 2024. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Official Journal of the EU, OJ L 2024/1689.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1705.08500. arXiv:1705.08500.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. URL https://arxiv.org/abs/2004.07780. arXiv:2004.07780.

Ole-Christoffer Granmo. The Tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic. *arXiv preprint arXiv:1804.01508*, 2018. URL https://arxiv.org/abs/1804.01508.

Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. In *NeurIPS Datasets and Benchmarks*, 2024. URL https://arxiv.org/abs/2406.18495. arXiv:2406.18495.

Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing (SAT)*, 2018. URL https://link.springer.com/chapter/10.1007/978-3-319-94144-8_26.

Alexey Ignatiev, Nina Narodytska, and Joao Marques-Silva. Abduction-based explanations for machine learning models. In *AAAI Conference on Artificial Intelligence*, 2019. URL https://ojs.aaai.org/index.php/AAAI/article/view/3964.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023. URL https://arxiv.org/abs/2312.06674.

Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *Computer Aided Verification (CAV)*, 2017. URL https://arxiv.org/abs/1702.01135. arXiv:1702.01135.

Hugo Krawczyk, Mihir Bellare, and Ran Canetti. HMAC: Keyed-hashing for message authentication (rfc 2104), 1997. URL https://datatracker.ietf.org/doc/html/rfc2104. IETF RFC 2104.

Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency (rfc 6962), 2013. URL https://datatracker.ietf.org/doc/html/rfc6962. IETF RFC 6962.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011.

Tom McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. URL https://arxiv.org/abs/1902.01007. arXiv:1902.01007.

Nina Narodytska, Shiva Kasiviswanathan, Leonid Ryzhyk, Mooly Sagiv, and Toby Walsh. Verifying properties of binarized deep neural networks. In *AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1709.06662. arXiv:1709.06662.

Emilia Przybysz, Bimal Bhattarai, Cosimo Persia, Ana Ozaki, Ole-Christoffer Granmo, and Jivitesh Sharma. Verifying properties of Tsetlin machines. In *International Symposium on the Tsetlin Machine (ISTM)*, 2023. URL https://arxiv.org/abs/2303.14464. arXiv:2303.14464; IEEE Xplore 10454997.

Bruce Schneier and John Kelsey. Cryptographic support for secure logs on untrusted machines. In *USENIX Security Symposium*, 1998.

Mrinank Sharma et al. Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv preprint arXiv:2501.18837*, 2025. URL https://arxiv.org/abs/2501.18837.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *ASPLOS*, 2006. URL https://dl.acm.org/doi/10.1145/1168857.1168907.

G. S. Tseitin. On the complexity of derivation in propositional calculus. In *Automation of Reasoning*, pages 466–483. Springer, 1983.

Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing (SAT)*, 2014. URL https://link.springer.com/chapter/10.1007/978-3-319-09284-3_31.

Rohan Kumar Yadav, Lei Jiao, Ole-Christoffer Granmo, and Morten Goodwin. Human-level interpretable learning for aspect-based sentiment analysis (and interpretable Tsetlin-machine text classification). *arXiv preprint arXiv:2105.04708*, 2021. URL https://arxiv.org/abs/2105.04708.

Wenjun Zeng et al. ShieldGemma: Generative AI content moderation based on Gemma. *arXiv preprint arXiv:2407.21772*, 2024. URL https://arxiv.org/abs/2407.21772.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023. URL https://arxiv.org/abs/2307.15043.